

```

1. MOVE 10, [a] ;
2. MOVE 11, [b] ;
3. ADD 13, 10, 10;
4. ADD 12, 10, 11;
5. ADD 14, 10, 11;
6. MOVE 15, 14 ;
7. ADD 16, 13, 13;

```

Using the method described above, we construct the Interference Graph for the code, and attempt to colour it, as shown in Figure 8. This interference graph is un-colourable because the maximum number of available registers is three (*R0*, *R1* and *R2*), and to colour node *I3*, we would require another register *R3*, which is not available.

To work around this situation, we need to generate spill-code—i.e., we spill (write) a register to memory. For this, we need to insert two memory transfer instructions, one to write the value of the register to memory, and the other to read the value into the register from memory. For our example, we can generate spill-code like that at Instructions 4 and 8 in the following code:

```

1. MOVE 10, [a] ;
2. MOVE 11, [b] ;
3. ADD 13, 10, 10;
4. MOVE [c], 13 ; //Spill register to memory
5. ADD 12, 10, 11;
6. ADD 14, 10, 11;
7. MOVE 15, 14 ;
8. MOVE 17, [c] ; //Read from memory location
9. ADD 16, 17, 17; //Replace register 13 by 17

```

Here, we are spilling Register 13 to memory. The register to be spilled is known as the *spill-candidate*. The spilling of a register (like *I3*) to memory (say, location *[c]*) should be done just after the instruction in which Register *I3* was defined. Reading the value back from the memory location should be done just before the instruction in which Register *I3* was used—but note that we need to introduce a new virtual register (say, *I7*), to read from the memory location *[c]* and accordingly, replace the use of virtual Register *I3* in Instruction 9 with Register *I7*.

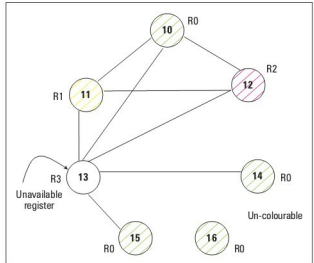


Figure 8: An un-colourable Interference Graph

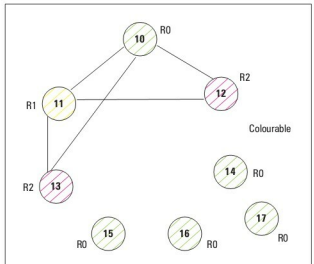


Figure 9: A colourable Interference Graph after spilling to memory

With this modification, we can now colour all the nodes of the graph with the three given registers, as shown in Figure 9.

Thus, generating spill-code has led to the successful colouring of the Interference Graph. However, this might not always be beneficial, because the addition of spill-code implies the addition of two memory transfer instructions. In some cases, we might be able to combine the generated spill-code instructions in parallel with some other instruction, which will prove to be the best solution. 

By: Deepti Sharma

The author is a senior software engineer (Team Lead) at Arme Technologies Pvt Ltd. She has a considerable amount of experience in the development of compilers and tools for embedded systems. She can be reached at deepti.acmet@gmail.com.